EKS




Installation of Tools

For Ubuntu / Mac

- Brew: https://brew.sh/

For Windows

- Chocolatey: https://chocolatey.org/install

Installation of Dependancies

For Brew

- Brew install awscli
- brew install eksctl
- brew install kubectl
- brew install helm

For Chocolatey

- choco install awscli
- choco install eksctl
- choco install kubernetes-cli
- choco install helm

What is EKS

Amazon EKS (Elastic Kubernetes Service) is a fully managed Kubernetes service by AWS. It simplifies the deployment, management, and scaling of containerized applications using Kubernetes. Key features include:

Fully Managed: AWS handles Kubernetes control plane management.

Scalable & Reliable: Integrates with AWS services like EC2, ELB, and IAM.

Security: Leverages AWS security features for access control and networking.

Easy Integration: Seamlessly integrates with other AWS services like CloudWatch and Route 53.

EKS lets you focus on building applications while AWS handles infrastructure management.

Types of EKS Cluster

Self-Managed Nodes:

You manage the EC2 instances (nodes) in your EKS cluster.

You are responsible for provisioning, scaling, and maintaining these nodes.

You can choose any instance type and configure the nodes as per your needs.

Managed Nodes:

AWS automatically manages the EC2 instances for you.

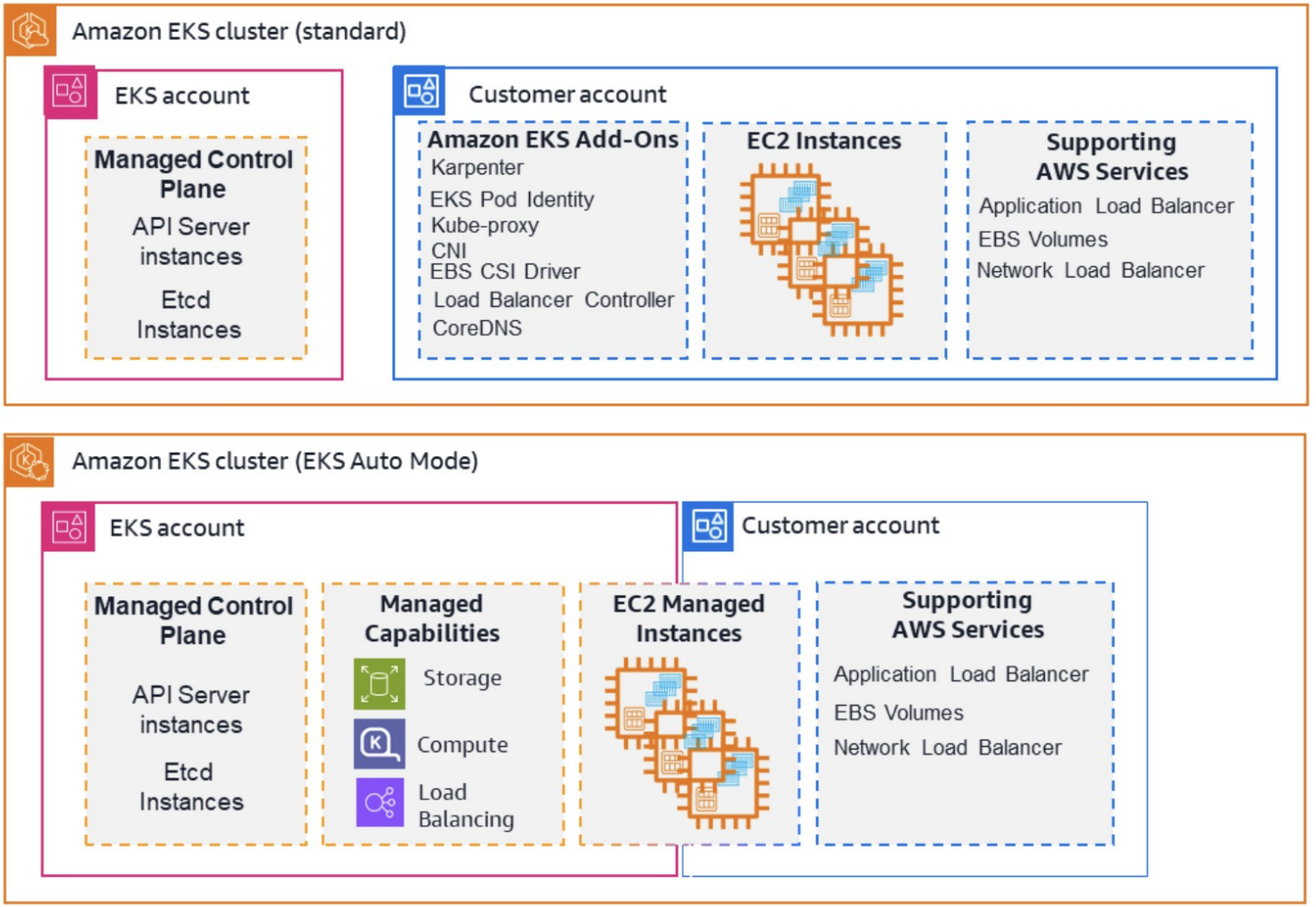
AWS handles scaling, patching, and the underlying infrastructure for the worker nodes.

This is easier to set up and maintain, with AWS ensuring your worker nodes are always up-to-date.

Amazon EKS on Fargate:

Allows running Kubernetes pods without managing the underlying EC2 instances.

AWS automatically provisions and scales the compute resources based on demand.


Amazon EKS cluster (standard)
EKS account
Managed Control Plane
API Server instances
Etcd Instances
Customer account
Amazon EKS Add-Ons
Karpenter
EKS Pod Identity
Kube-proxy
CNI
EBS CSI Driver
Load Balancer Controller
CoreDNS
EC2 Instances
Supporting AWS Services
Application Load Balancer
EBS Volumes
Network Load Balancer
Amazon EKS cluster (EKS Auto Mode)
EKS account
Managed Control Plane
API Server instances
Etcd Instances
Managed Capabilities
Storage
Compute
Load Balancing
EC2 Managed Instances
Customer account
Supporting AWS Services
Application Load Balancer
EBS Volumes
Network Load Balancer

eksctl

To create cluster

eksctl create cluster --name devops --region us-east-1 --fargate --version 1.32

To update kube-conf

aws eks update-kubeconfig --name devops --region us-east-1

To delete clustereksctl

delete cluster --name devops --region us-east-1

IAM OIDC Provider

- It is an entity that AWS trusts to authenticate users or applications using the OpenID Connect (OIDC) protocol, allowing you to grant access to AWS resources based on the identity provided by the OIDC provider

Commands to configure IAM OIDC provider

export cluster_name=devops

oidc_id=$(aws eks describe-cluster --name $cluster_name --region us-east-1 --query "cluster.identity.oidc.issuer" --output text | cut -d '/' -f 5)

Check if there is an IAM OIDC provider configured

eksctl utils associate-iam-oidc-provider --cluster $cluster_name --region us-east-1 --approve

AWS Load Balancer Controller

- The AWS Load Balancer Controller is a Kubernetes controller that manages AWS Elastic Load Balancers (ALBs and NLBs) for a Kubernetes cluster, allowing you to expose your cluster apps to the internet using Kubernetes Ingress resources or Services of type LoadBalancer.

How to setup alb

Download IAM policy

curl -O https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-controller/v2.11.0/docs/install/iam_policy.json

Create IAM Policy

aws iam create-policy \ --policy-name AWSLoadBalancerControllerIAMPolicy \ --policy-document file://iam_policy.json

Create IAM Role

eksctl create iamserviceaccount \ --cluster=<your-cluster-name> \ --region=<aws-region> \ --namespace=kube-system \ --name=aws-load-balancer-controller \ --role-name AmazonEKSLoadBalancerControllerRole \ --attach-policy-arn=arn:aws:iam::<your-aws-account-id>:policy/AWSLoadBalancerControllerIAMPolicy \ --approve

Add helm repo

helm repo add eks https://aws.github.io/eks-charts

Update the repo

helm repo update eks

Install aws-alb-ingress-controller

helm install aws-load-balancer-controller eks/aws-load-balancer-controller -n kube-system \ --set clusterName=<your-cluster-name> \ --set serviceAccount.create=false \ --set serviceAccount.name=aws-load-balancer-controller \ --set region=<region> \ --set vpcId=<your-vpc-id>

Verify that the deployments are running.

kubectl get deployment -n kube-system aws-load-balancer-controller

Application Deployment

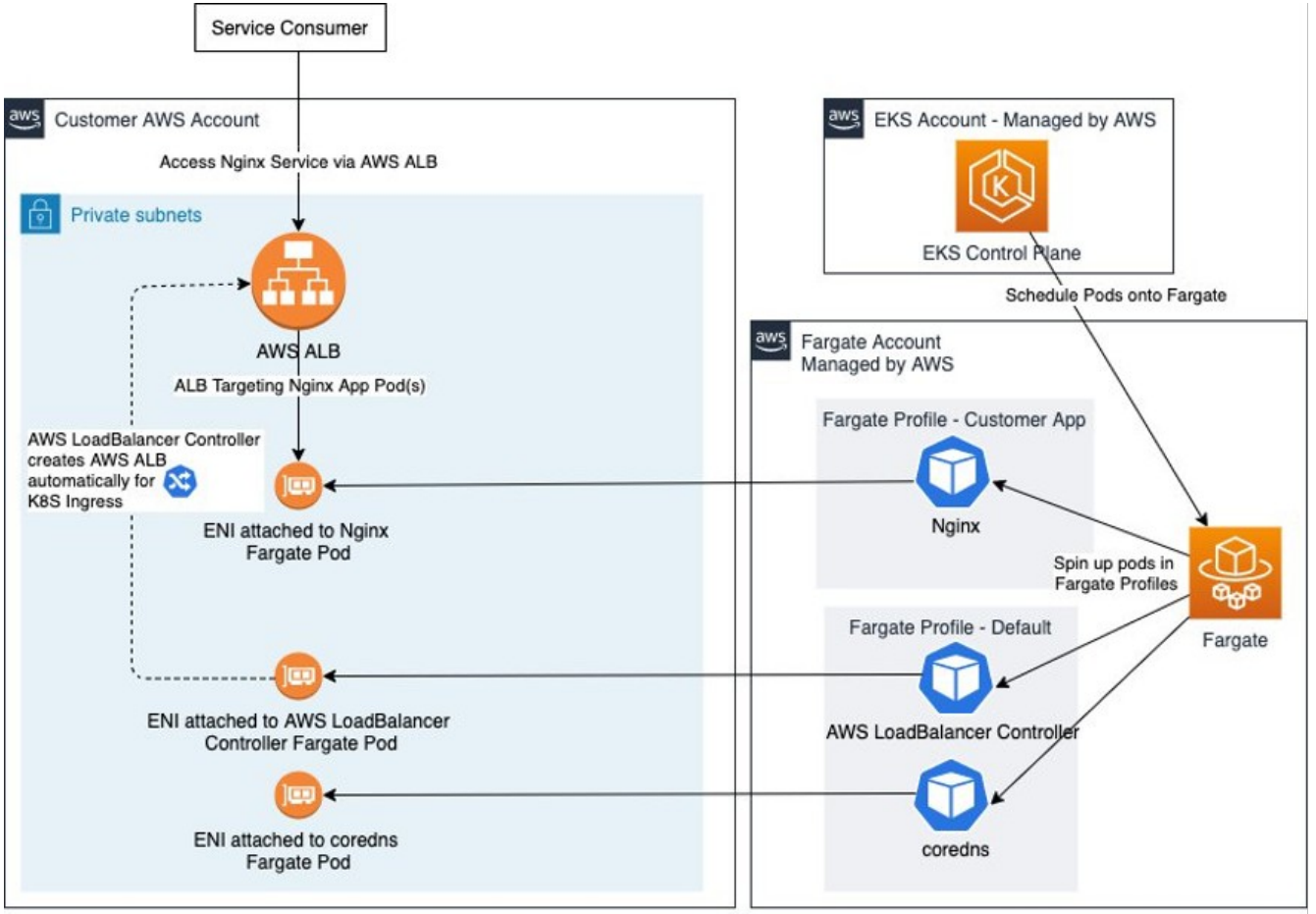
Node Affinity

- It is a set of rules that determines which nodes a pod can be scheduled on, based on labels on those nodes.
- It allows you to express preferences or requirements for where a pod should run, based on node labels

Pod Affinity

- Pod affinity provides control over the placement of pods relative to each other, enabling you to group pods together for co-location or separation for specific needs.
- 2 Types of Affinity:

- Pod Affinity: Specifies that certain pods should be placed as close as possible to other groups of pods.
- Pod Anti-Affinity: Specifies that certain pods should be kept as far apart as possible from other groups of pods.



Architecture Overview - Nginx App on EKS Fargate fronted by ALB

```
2   apiVersion: v1
3   kind: Namespace
4   metadata:
5     name: game-2048
```

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  namespace: game-2048
  name: deployment-2048
spec:
  selector:
    matchLabels:
      app.kubernetes.io/name: app-2048
  replicas: 5
  template:
    metadata:
      labels:
        app.kubernetes.io/name: app-2048
    spec:
      containers:
      - image: public.ecr.aws/l6m2t8p7/docker-2048:latest
        imagePullPolicy: Always
        name: app-2048
        ports:
        - containerPort: 80
```

```yaml
apiVersion: v1
kind: Service
metadata:
  namespace: game-2048
  name: service-2048
spec:
  ports:
    - port: 80
      targetPort: 80
      protocol: TCP
  type: NodePort
  selector:
    app.kubernetes.io/name: app-2048
```

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  namespace: game-2048
  name: ingress-2048
  annotations:
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
spec:
  ingressClassName: alb
  rules:
    - http:
        paths:
        - path: /
          pathType: Prefix
          backend:
            service:
              name: service-2048
              port:
                number: 80
```

Deployment

- *eksctl create fargateprofile --cluster devops --region us-east-1 --name 2048-app --namespace game-2048*
- *kubectl apply -f https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-controller/v2.5.4/docs/examples/2048/2048_full.yaml*
- *kubectl get pods -n game-2048*
- *kubectl get svc -n game-2048*
- *kubectl get ingress -n game-2048*

Observability

Namespace

- A mechanism for logically partitioning resources within a cluster, allowing multiple teams or projects to share the same cluster without interfering with each other.
- Namespaces provide a scope for Kubernetes resources, enabling resource isolation and management.

create fargateprofile for monitoring namespace

```
eksctl create fargateprofile \
    --cluster devops \
    --region us-east-1 \
    --name observability \
    --namespace monitoring
```

To create namespace

```
kubectl create namespace monitoring
```

Installation of Grafana

To add the Grafana repository,

helm repo add grafana https://grafana.github.io/helm-charts

To update Repo

helm repo update

To install grafana with load balancer service

helm install grafana grafana/grafana --namespace monitoring

Get pods

kubectl get pods -n monitoring

To get grafana default password

kubectl get secret --namespace monitoring grafana -o jsonpath="{.data.admin-password}" | base64 --decode ; echo

Get all service in monitoring

kubectl get svc -n monitoring

To create ingress for Grafana

grafana-ingress.yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  namespace: monitoring
  name: grafana-ingress
  annotations:
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
spec:
  ingressClassName: alb
  rules:
    - http:
        paths:
        - path: /
          pathType: Prefix
          backend:
            service:
              name: grafana
              port:
                number: 80

```

kubectl apply -f grafana-ingress.yml

kubectl get ingress -n monitoring

Installing Prometheus

Installing prometheus

helm install prometheus prometheus-community/prometheus --namespace monitoring

kubectl get deployment.apps/prometheus-server -n monitoring -o yaml

Debug the deployment

kubectl edit deployment.apps/prometheus-server -n monitoring

```
#line no 141 where volume:  replace by  emptyDir: {}
"spec": {
    "template": {
      "spec": {
        "volumes": [
          {
            "name": "storage-volume",
            "emptyDir": {}
          },
```

To create ingress for Prometheus

prometheus-ingress.yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  namespace: monitoring
  name: prometheus-ingress
  annotations:
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
spec:
  ingressClassName: alb
  rules:
    - http:
        paths:
        - path: /
          pathType: Prefix
          backend:
            service:
              name: prometheus-server
              port:
                number: 80

```

kubectl apply -f prometheus-ingress.yaml

kubectl get ingress -n monitoring

To delete cluster

eksctl delete cluster --name devops --region us-east-1

ECR & ECS

ECR

- A fully managed container registry service provided by Amazon Web Services (AWS).
- Secure, scalable, and reliable.
- Supports private and public repositories.
- Integrates with Docker CLI for pushing, pulling, and managing images.
- Offers lifecycle policies for managing image lifecycles.
- Provides image scanning for identifying vulnerabilities.
- Supports replication across regions.

What is the difference between Amazon ECR public and private repositories?

A private repository does not offer content search capabilities and requires Amazon IAM-based authentication using AWS account credentials before allowing images to be pulled. A public repository has descriptive content and allows anyone anywhere to pull images without needing an AWS account or using IAM credentials. Public repository images are also available in the Amazon ECR public gallery.

Push commands for devops/eks

macOS / Linux | Windows

Make sure that you have the latest version of the AWS CLI and Docker installed. For more information, see Getting started with Amazon ECR.

Use the following steps to authenticate and push an image to your repository. For additional registry authentication methods, including the Amazon ECR credential helper, see Registry authentication.

1. Retrieve an authentication token and authenticate your Docker client to your registry. Use the AWS CLI:

   ```
   aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 264152483755.dkr.ecr.us-east-1.amazonaws.com
   ```

   Note: if you receive an error using the AWS CLI, make sure that you have the latest version of the AWS CLI and Docker installed.

2. Build your Docker image using the following command. For information on building a Docker file from scratch, see the instructions here. You can skip this step if your image has already been built:

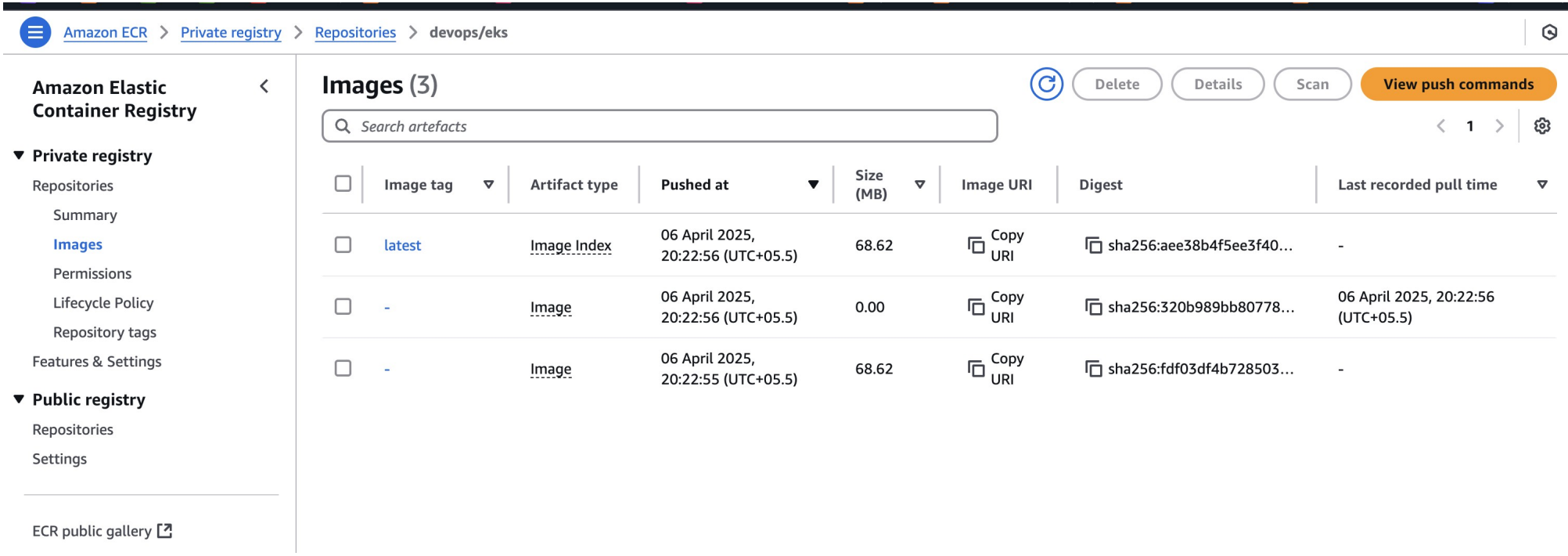
   ```
   docker build -t devops/eks .
   ```

3. After the build is completed, tag your image so you can push the image to this repository:

   ```
   docker tag devops/eks:latest 264152483755.dkr.ecr.us-east-1.amazonaws.com/devops/eks:latest
   ```

4. Run the following command to push this image to your newly created AWS repository:

   ```
   docker push 264152483755.dkr.ecr.us-east-1.amazonaws.com/devops/eks:latest
   ```


Amazon ECR > Private registry > Repositories > devops/eks
Amazon Elastic Container Registry
Private registry
Repositories
Summary
Images
Permissions
Lifecycle Policy
Repository tags
Features & Settings
Public registry
Repositories
Settings
ECR public gallery
Images (3)
Delete
Details
Scan
View push commands
Search artefacts
1
Image tag
Artifact type
Pushed at
Size (MB)
Image URI
Digest
Last recorded pull time
latest
Image Index
06 April 2025, 20:22:56 (UTC+05.5)
68.62
Copy URI
sha256:aee38b4f5ee3f40...
-
-
Image
06 April 2025, 20:22:56 (UTC+05.5)
0.00
Copy URI
sha256:320b989bb80778...
06 April 2025, 20:22:56 (UTC+05.5)
-
Image
06 April 2025, 20:22:55 (UTC+05.5)
68.62
Copy URI
sha256:fdf03df4b728503...
-

ECS

Amazon Elastic Container Service (ECS) is a fully managed container orchestration service that simplifies the deployment, management, and scaling of containerized applications on AWS, allowing you to run and manage Docker containers without the need to manage the underlying infrastructure.

Components of ECS

Task Definition:

- You define your container application using a task definition, specifying the Docker image, CPU/memory requirements, networking, and other parameters.

Cluster:

- You create an ECS cluster, which is a logical grouping of EC2 instances or Fargate tasks where your containers will run.

Services:

- You define services that manage the desired number of running tasks based on the task definition, ensuring your application is always available.

Scheduling:

- ECS automatically schedules your tasks across the cluster based on resource availability and your specified constraints.

Management:

- ECS provides tools for monitoring, logging, and managing your containerized applications.

Amazon Elastic Container Service

Clusters
Namespaces
Task definitions
Account settings

Install AWS Copilot

Amazon ECR
Repositories

AWS Batch

Documentation
Discover products
Subscriptions

Tell us what you think

test Info

Last updated
06 April 2025 at 20:29 (UTC+5:30)

Update service | Delete service

Service overview Info

Status	Tasks (1 Desired)	Task definition: revision	Deployment status
Active	1 pending | 0 running	devops-demo:1	In progress

Health and metrics | Tasks | Logs | Deployments | Events | Configuration and networking | Service auto scaling | Tags

Tasks (1/1)

Stop

Filter tasks by property or value | Filter desired status: Any desired status | Filter launch type: Any launch type

Task	Last status	Desired st...	T...	Health sta...	Started by	Started at
9c23e259525a4759982cb...	Pending	Running	dev...	Unknown	ecs-svc/86122309680...	-

Containers for task 9c23e259525a4759982cb55d490beaf8

Containers (1)

Filter containers

Container name	Container runtime ID	Image URI	Image Digest	Status	Health status	CPU
nginx	-	26415248...	-	Pending	Unknown	0